

Universidad
Rey Juan Carlos

ESCUELA TÉCNICA SUPERIOR
DE INGENIERÍA INFORMÁTICA

GRADO EN INTELIGENCIA ARTIFICIAL

Práctica 1: Detección de Objeto Plano y Realidad Aumentada

VISIÓN ARTIFICIAL

CURSO 2024-2025

**Redactado por: Carmen Maeso Soler
Daniel del Nuevo Montero**

Índice

1. Introducción	2
2. Detección de un objeto plano con un diseño particular	4
2.1. Cálculo de la homografía H_t^{img}	4
2.1.1. Detección de Puntos de Interés, Descriptores y Emparejamiento	4
2.1.2. Obtener de la homografía H_t^{img} con cv2.findHomography y RANSAC	5
2.2. Cálculo de la homografía H_w^{img}	6
2.3. Cálculo de R y t a partir de H_w^{img}	6
2.4. Realidad aumentada sobre la imagen	8
2.4.1. Proyección del sistema de referencia	8
2.4.2. Proyección del cubo 3D	9
3. Cálculo de H_t^{img} con el marco rojo de la plantilla	12
3.1. Umbralizado y procesamiento	13
3.2. Detección de contornos	15
3.2.1. Detección de contornos anidados	15

1. Introducción

El objetivo principal de esta práctica es detectar un objeto plano impreso en una hoja DIN A4, estimar su posición y orientación en el espacio tridimensional con respecto a una cámara conocida, y superponer un modelo 3D proporcionado en un lugar concreto de la plantilla.

Para ello, se utilizarán técnicas de visión artificial como la detección de puntos de interés, el cálculo de homografías y la estimación de la matriz de proyección de la cámara. Antes de comenzar, analizamos algunos aspectos sobre el material proporcionado:

- El objeto plano, representado por la imagen `template_cropped.png`, aparece en diferentes posiciones y orientaciones dentro de una secuencia de imágenes capturadas por una cámara con resolución de 4000×3000 píxeles. De dicha cámara se dispone la matriz de parámetros intrínsecos (`intrinsics.txt`).



Figura 1: Plantilla utilizada en la práctica (izq.) y una imagen de la misma impresa en papel (dcha.)

- Se proporciona un modelo 3D en formato `.obj` que debe colocarse correctamente sobre el plano detectado, ocupando una región específica de la plantilla.



Figura 2: Ejes del sistema de referencia 3D sobre el plano (esquina superior izquierda de la plantilla) y cubo 3D proyectado en la posición deseada.

- Se ofrecen dos conjuntos de imágenes: una secuencia de 28 imágenes con pequeños desplazamientos entre fotogramas, y una secuencia de test de 34 imágenes con mayor variabilidad de punto de vista y orientación.

La práctica se compone de tres partes: 1) detección basada en puntos clave y homografía, 2) detección utilizando el marco rojo exterior del objeto, y 4) una sección de exploración con mejoras o ideas alternativas. A lo largo de la memoria se explicará la metodología seguida, los algoritmos implementados y los resultados obtenidos, incluyendo las limitaciones encontradas en el desarrollo.

A continuación se muestra el esquema general de la práctica:

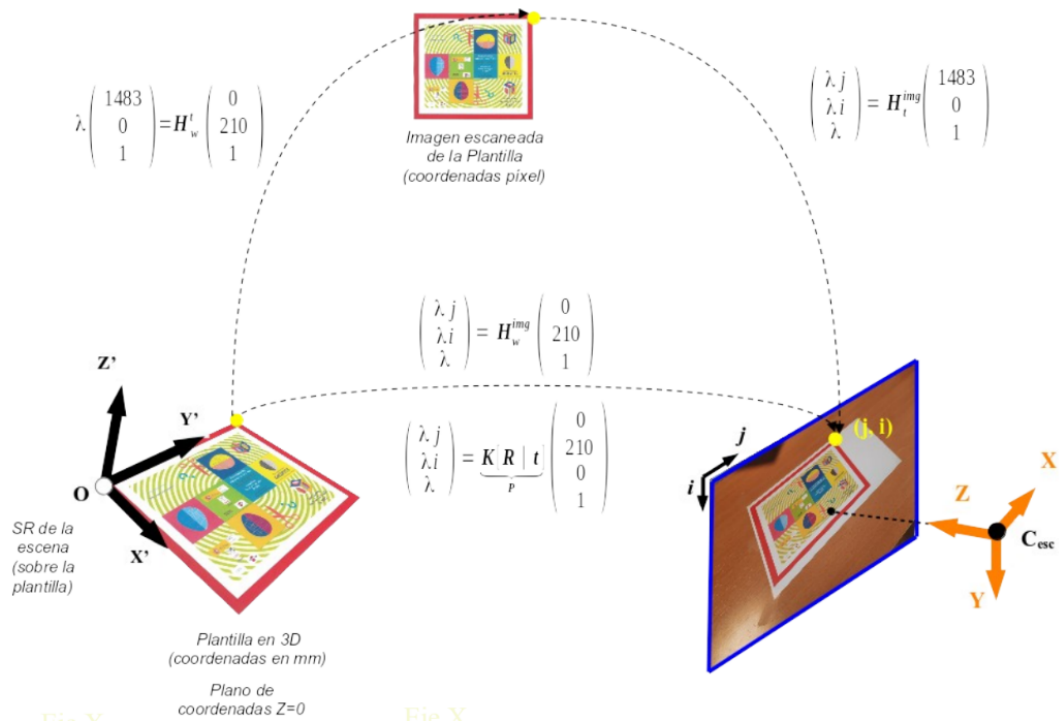


Figura 3: Detalle de la geometría del problema a resolver con las diferentes transformaciones desde el objeto real plano (coordenadas en mm) hasta la imagen de entrada (coordenadas píxel) pasando por la imagen de la plantilla escaneada (coordenadas píxel).

2. Detección de un objeto plano con un diseño particular

2.1. Cálculo de la homografía H_t^{img}

2.1.1. Detección de Puntos de Interés, Descriptores y Emparejamiento

En primer lugar hemos de detectar los puntos de interés (P.I.) de la imagen plantilla y de las de secuencia (imagen).

- Utilizando un **detector** $\in \{\text{SIFT}, \text{ORB}\}$, hemos obtenido:
 - Los puntos de interés de la plantilla: $\text{P.I.}_{\text{detector}}(\text{plantilla})$.
 - Los puntos de interés de la imagen: $\text{P.I.}_{\text{detector}}(\text{imagen})$.
 - Los descriptores correspondientes: $\text{DESC}_{\text{detector}}(\text{plantilla})$ y $\text{DESC}_{\text{detector}}(\text{imagen})$.
- A continuación, se emparejan los puntos de interés detectados en la imagen y en la plantilla:
 - $\text{P.I.}_{\text{detector}}(\text{imagen})$ con $\text{P.I.}_{\text{detector}}(\text{plantilla})$,
 - en base a los descriptores: $\text{DESC}_{\text{detector}}(\text{imagen})$, $\text{DESC}_{\text{detector}}(\text{plantilla})$.
- Para el emparejamiento se utilizan diferentes técnicas, dependiendo del tipo de descriptores generados por el detector:
 - En el caso de **SIFT**, los descriptores están en *coma flotante*.
 - En el caso de **ORB**, los descriptores están en *formato binario*.
 - Los métodos utilizados para realizar el emparejamiento son:
 - **Brute-Force matcher:** compara cada descriptor de la plantilla con todos los de la imagen. Dependiendo del tipo de descriptor, se usa una métrica diferente:
 - ◇ Para **SIFT**: `cv2.NORM_L2`.
 - ◇ Para **ORB**: `cv2.NORM_HAMMING`.Más información disponible en: [BFMatcher - Documentación OpenCV](#)
 - **FLANN Based Matcher:** utiliza estructuras optimizadas para la búsqueda eficiente del vecino más cercano. Es especialmente útil con grandes volúmenes de datos. Más información en: [FLANN - Documentación OpenCV](#)

- Además, se aplica un filtrado adicional propuesto por **David Lowe** en su artículo “*Distinctive image features from scale-invariant keypoints*”¹. Este filtrado consiste en conservar solamente aquellos emparejamientos donde la relación de distancias entre el mejor y el segundo mejor vecino sea menor que un umbral (habitualmente 0.75), lo que mejora la robustez ante emparejamientos incorrectos.
- En la figura de abajo podemos observar los **matches** obtenidos mediante Brute-Force matching y FLANN-Based matching para los P.I. detectados tanto con SIFT como con ORB. Concluimos que **SIFT y FLANN suelen ser más robustos cuando hay muchos puntos de interés y presencia de textura**, ya que SIFT produce descriptores más informativos y FLANN permite hacer búsquedas rápidas en espacios de alta dimensión.

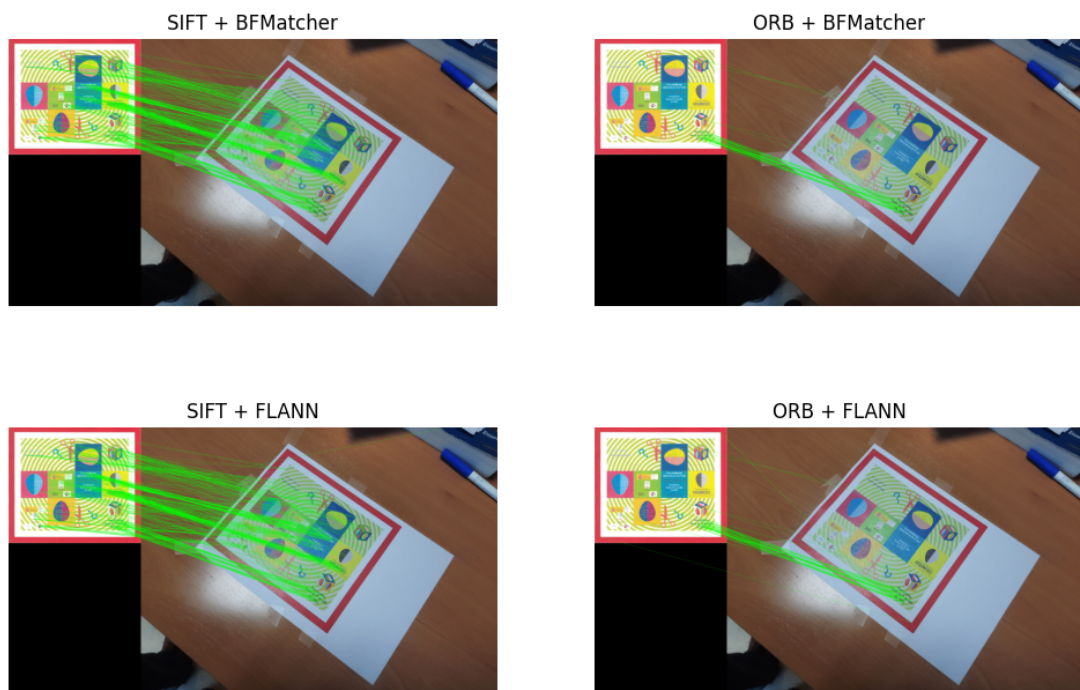


Figura 4: Matches obtenidos con las diferentes técnicas.

2.1.2. Obtener de la homografía H_t^{img} con `cv2.findHomography` y RANSAC

Ahora, procedemos a obtener la matriz de homografía, para lo cual utilizaremos ‘`cv2.findHomography`’ y el algoritmo RANSAC: - En este contexto, RANSAC se utiliza para excluir los *outliers* de los *matches* (por ejemplo, podemos observar que en ocasiones se empareja un punto del template con cierto punto de la mesa o de la escena en general en lugar de con el punto al que correspondería realmente).

¹[Google Scholar - David Lowe](#)

2.2. Cálculo de la homografía H_w^{img}

Para calcular la homografía H_w^{img} , que transforma coordenadas del sistema de referencia (SR) del objeto en el mundo real (en milímetros) al SR de la imagen, primero debemos obtener la homografía intermedia H_w^t , que lleva del SR del objeto real en mm a coordenadas en píxeles de la plantilla escaneada (*template*).

- Declaramos las 4 esquinas del objeto respecto a su propio SR en milímetros en el siguiente orden [*top-left*, *top-right*, *down-right*, *down-left*], teniendo en cuenta los siguientes datos proporcionados por el archivo `resources/MedidasPlantilla.pdf` :
 - `ancho(template) = 210 mm`.
 - `alto(template) = 185 mm`.
- Declaramos con el mismo orden 4 esquinas del objeto en la plantilla escaneada en coordenadas de píxeles. Para ello, utilizamos el atributo `shape` de NumPy sobre la imagen cargada de la plantilla.
- Utilizamos la función `cv2.getPerspectiveTransform` de OpenCV para obtener la homografía H_w^t .
- Una vez tenemos la homografía H_w^t , podemos obtener la homografía H_w^{img} mediante el siguiente cálculo, que corresponde a una multiplicación punto a punto (producto matricial):

$$H_w^{\text{img}} = H_t^{\text{img}} \cdot H_w^t$$

2.3. Cálculo de R y t a partir de H_w^{img}

Para poder mostrar los ejes del sistema de referencia (SR) de la plantilla en la escena y proyectar modelos 3D sobre ella, es necesario obtener la **matriz de proyección P** , que transforma coordenadas de un punto 3D (respecto al SR del objeto, en coordenadas homogéneas) en coordenadas 2D respecto al SR de la imagen.

La matriz de proyección P se obtiene mediante:

$$P = K[R \mid t]$$

- K es la **matriz de parámetros intrínsecos** de la cámara, proporcionada en los archivos `data/secuencia/intrinsics.txt` y `data/test/intrinsics.txt`.

- $\mathbf{R}$ es la **matriz de rotación**, que describe la orientación del SR de la plantilla respecto a la cámara.
- $\mathbf{t}$ es el **vector de traslación**, que indica el desplazamiento del origen del SR de la plantilla en la escena.

Para obtener $\mathbf{R}$ y $\mathbf{t}$, partimos de la homografía $\mathbf{H}_w^{\text{img}}$ y seguimos los siguientes pasos:

- Calculamos $\mathbf{H}^* = \mathbf{K}^{-1} \mathbf{H}_w^{\text{img}}$:

$$\mathbf{H}^* = \begin{pmatrix} \underbrace{\mathbf{h}_1}_{\begin{smallmatrix} h_{11}^* \\ h_{21}^* \\ h_{31}^* \end{smallmatrix}} & \underbrace{\mathbf{h}_2}_{\begin{smallmatrix} h_{12}^* \\ h_{22}^* \\ h_{32}^* \end{smallmatrix}} & \underbrace{\mathbf{h}_3}_{\begin{smallmatrix} h_{13}^* \\ h_{23}^* \\ h_{33}^* \end{smallmatrix}} \end{pmatrix}$$

- Calculamos el factor de escala: $\lambda = \|\mathbf{h}_1\|$.
- Obtenemos los vectores columna de la matriz de rotación:

$$\begin{aligned} \mathbf{r}_1 &= \frac{\mathbf{h}_1}{\lambda} \\ \mathbf{r}_2 &= \frac{\mathbf{h}_2}{\lambda} \\ \mathbf{r}_3 &= \mathbf{r}_1 \times \mathbf{r}_2 \end{aligned}$$

- Finalmente, obtenemos el vector de traslación:

$$\mathbf{t} = \frac{\mathbf{h}_3}{\lambda}$$

2.4. Realidad aumentada sobre la imagen

2.4.1. Proyección del sistema de referencia

Finalmente, teniendo la matriz de proyección $\mathbf{P}$, podemos obtener las coordenadas de las rectas que componen la visualización del sistema de referencia (SR) en las escenas:

- Establecemos la longitud de los ejes, a la que llamamos l (por ejemplo, `axs_length` en el código).
- Definimos los siguientes 4 puntos en **coordenadas homogéneas**:
 - Origen de coordenadas: $(0 \ 0 \ 0 \ 1)^\top$
 - Punto sobre el eje X: $(l \ 0 \ 0 \ 1)^\top$
 - Punto sobre el eje Y: $(0 \ l \ 0 \ 1)^\top$
 - Punto sobre el eje Z: $(0 \ 0 \ l \ 1)^\top$
 - A continuación, proyectamos estos puntos 3D sobre la imagen utilizando la matriz de proyección $\mathbf{P}$. Esto se realiza mediante la multiplicación $\mathbf{p} = \mathbf{P} \cdot \mathbf{X}$, seguida de la normalización homogénea. El siguiente código muestra esta operación:

```

1 def get_points2D(points3D: np.ndarray, P: np.ndarray) ->
  ↪ np.ndarray:
2     points2D = P @ points3D
```

Finalmente, se representan las proyecciones en la imagen uniendo el origen con los extremos de cada eje mediante líneas rojas (X), verdes (Y) y azules (Z). Esto se realiza con la función `plotSR`:

Se han adjuntado todas las proyecciones para observar lo que ya advertíamos anteriormente: en general, ORB y BF muestran peores resultados para generar la homografía por este método, por lo que nos quedaríamos con SIFT y FLANN.

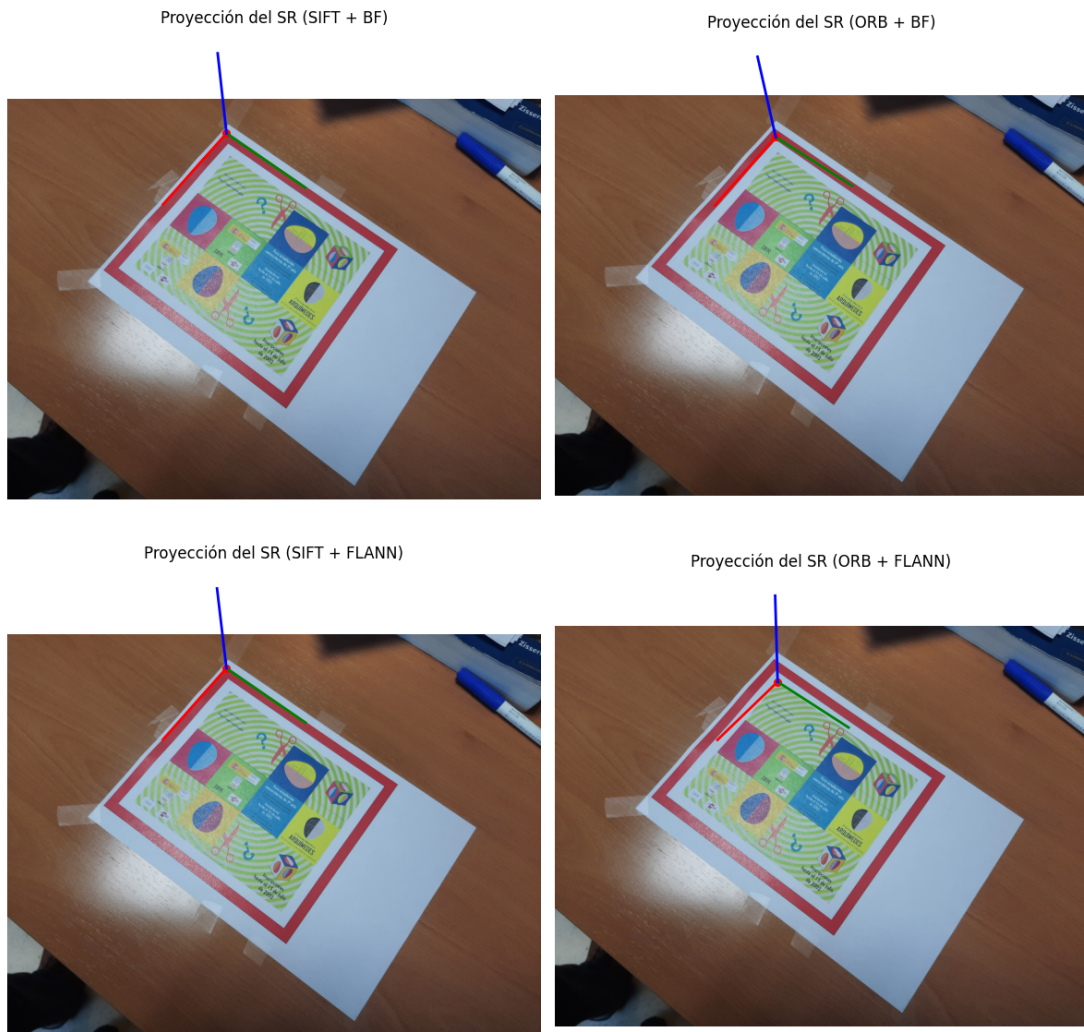


Figura 5: Ejes de coordenadas dibujados a partir de las distintas matrices de proyección obtenidas.

2.4.2. Proyección del cubo 3D

A continuación, se nos requiere **colocar un modelo tridimensional** (archivo .obj) que representa un cubo de **43 mm de lado** sobre la escena, alineado con el cuadrado azul claro presente en la plantilla.

- Las coordenadas de las esquinas de dicha región, definidas en el sistema de referencia del objeto plano (en milímetros), son las siguientes:

- **Esquina superior izquierda:** $(105 \ 71 \ 1)^T$
- **Esquina superior derecha:** $(148 \ 71 \ 1)^T$
- **Esquina inferior derecha:** $(148 \ 114 \ 1)^T$
- **Esquina inferior izquierda:** $(105 \ 114 \ 1)^T$



Figura 6: Coordenadas objetivo marcadas en la plantilla original.

Para colocar correctamente el cubo 3D sobre el plano, primero cargamos el archivo .obj utilizando la clase Model3D, proporcionada en el material de la práctica.

Una vez instanciado el modelo, podemos examinar su estructura geométrica utilizando algunos de los métodos de la clase, como .vertices, .faces y .face_centroids. A continuación se muestra un resumen de los datos obtenidos:

■ **Vértices del cubo** (formato numpy.ndarray de 8 vértices en coordenadas 3D):

```
1  [[ 1.  1. -1.]
2   [ 1. -1.  1.]
3   [ 1. -1. -1.]
4   [ 1.  1.  1.]
5   [-1. -1. -1.]
6   [-1.  1. -1.]
7   [-1.  1.  1.]
```

Caras del cubo (índices de los vértices que componen cada una de las 12 caras triangulares):

```
1  [[1 2 3]
2   [2 1 4]
3   [1 5 6]
4   [5 1 3]
5   [7 1 6]
6   [1 7 4]
7   [2 7 8]
8   [7 2 4]
9   [2 5 3]
10  [5 2 8]
11  [7 5 8]
```

Resumen:

- Total de vértices: 8
- Total de caras: 12

Como podemos observar, el cubo se encuentra centrado en $(0 \ 0 \ 0)^T$ y la distancia entre vértices consecutivos es de 2 unidades (de -1 a 1 en cada eje). Gracias a esta información, podemos obtener el **factor de escala** necesario para ajustar el cubo al tamaño real deseado (43 mm de lado) dividiendo el lado real entre el lado original:

$$\text{escala} = \frac{43}{2} = 21,5$$

Tras calcular la escala, escalamos el cubo utilizando el método `scale()` de la clase `Model3D`.

También debemos trasladar el modelo 3D a su posición correcta. Para ello, debemos tener en cuenta que el cubo tiene inicialmente el centro de su base en el punto

$$(0 \ 0 \ -1).$$

Sin embargo, tras aplicar el escalado con un factor de 21,5, ese punto se encuentra en

$$(0 \ 0 \ -21,5).$$

El centro de la región de la plantilla donde debe colocarse el cubo (el cuadrado azul claro) se encuentra en:

$$\left(\frac{148+105}{2} \ \frac{114+71}{2} \ 0\right) = (126,5 \ 92,5 \ 0),$$

en milímetros y respecto al sistema de referencia de la plantilla.

Teniendo en cuenta la posición del centro de la base del cubo en su SR y el centro de la región objetivo en el SR de la plantilla, la traslación que se debe aplicar para alinear correctamente el modelo es:

$$\mathbf{t} = (126,5 + 0 \ 92,5 + 0 \ 0 + 21,5) = (126,5 \ 92,5 \ 21,5).$$

Tras definir el vector de traslación como:

```
1 t = np.array([126.5, 92.5, -21.5]).reshape(1, 3)
   utilizamos el método translate(t) de la clase Model3D para aplicar dicha traslación
   al modelo 3D del cubo.
```

Finalmente, proyectamos el cubo sobre la imagen de la escena mediante el método `plot_on_image()`, utilizando las distintas matrices de proyección **P** obtenidas con los diferentes métodos explicados previamente (combinaciones de detectores y emparejadores: SIFT + BF, ORB + BF, SIFT + FLANN, ORB + FLANN).

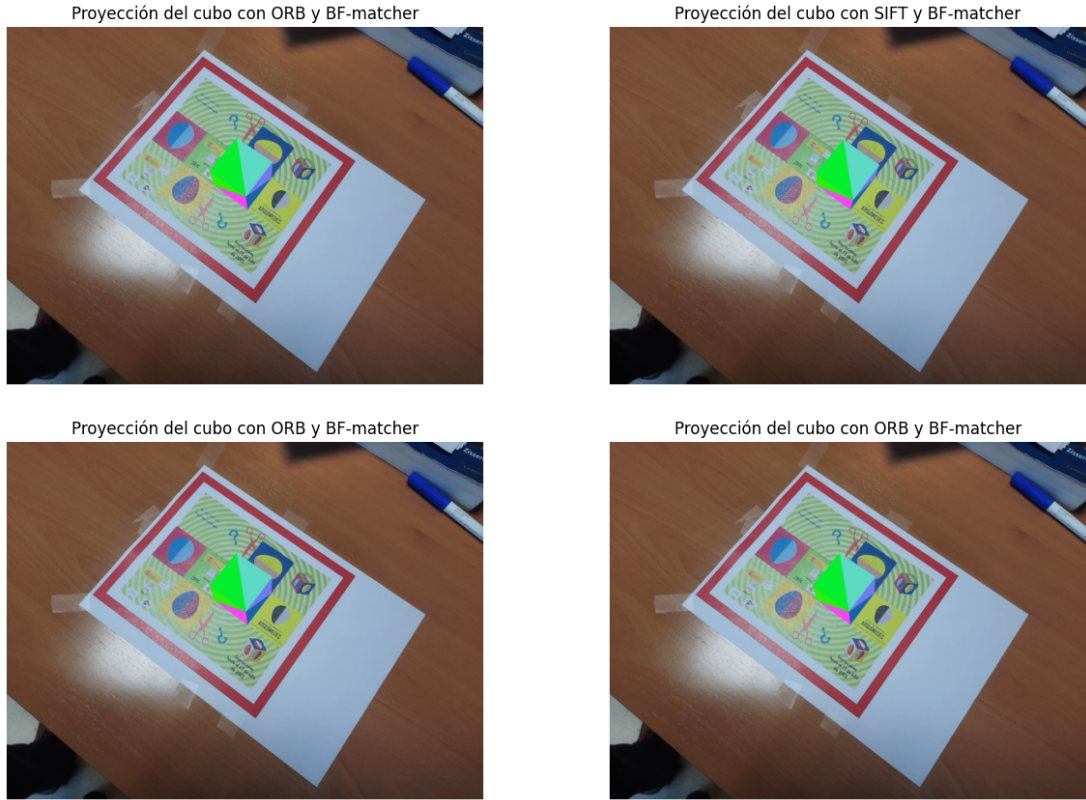


Figura 7: Proyección del cubo 3D sobre la plantilla utilizando diferentes combinaciones de detectores y emparejadores.

3. Cálculo de H_t^{img} con el marco rojo de la plantilla

En este apartado se solicita recalcular la homografía H_t^{img} , pero esta vez utilizando las esquinas exteriores del marco rojo que rodea la plantilla. El enunciado sugiere una posible solución basada en técnicas de procesamiento de imagen: detectar los píxeles correspondientes al color rojo mediante un umbral en el espacio de color, aplicar detección de contornos con `cv2.findContours`, simplificar los contornos a 4 vértices utilizando `cv2.approxPolyDP` (algoritmo de Ramer–Douglas–Peucker), y seleccionar aquellos contornos que compartan el mismo centro geométrico.

Esta fue la estrategia que intentamos implementar en primera instancia. Los pasos de umbralizado, detección de contornos y simplificación a polígonos de 4 vértices fueron suficientes para detectar las esquinas del marco rojo en la plantilla (aunque también podría haberse utilizado directamente el tamaño de la imagen con `.shape` para este caso concreto).

El principal problema surgió al intentar definir un umbral de color que permitiera detectar el rojo del marco de forma fiable y sin ruido, especialmente ante imágenes con condiciones de iluminación muy variables. Probamos varios enfoques:

- Umbralizado directo en RGB.
- Umbralizado adaptativo.
- Máscara de "predominancia de rojo", que selecciona píxeles con valor alto en el canal rojo y valores relativamente bajos en los canales verde y azul.

Sin embargo, ninguno de estos métodos fue suficientemente robusto para funcionar correctamente en todas las imágenes de la secuencia.

Por este motivo, optamos por cambiar del espacio de color RGB al espacio HSV. Esta decisión se basa en que HSV permite separar más claramente la componente de color (tono o *hue*) de la iluminación (valor o *value*), lo cual lo hace más adecuado para tareas de segmentación cromática bajo condiciones de iluminación cambiantes, aunque tampoco conseguimos detectar los contornos con este enfoque.

Esta fase fue la más laboriosa del desarrollo, ya que experimentamos con distintos *pipelines* de preprocesado, variaciones de umbrales y operaciones morfológicas como cierre, apertura, erosión y dilatación.

Finalmente, el *pipeline* seleccionado fue el siguiente:

Suavizado Gaussiano + Contraste Rojo + Adaptive Threshold +
Erosión + Apertura + Inversión

Aunque este flujo de trabajo produjo buenos resultados en algunas imágenes, no fue lo suficientemente robusto como para generalizar correctamente a todas las variaciones de iluminación presentes en las escenas.

3.1. Umbralizado y procesamiento

Todo este procesamiento lo encapsulamos en una clase llamada `ImagePreprocessor`. Está diseñada de forma modular para organizar claramente las operaciones aplicadas en el pipeline de segmentación del marco rojo. Al inicializarse, la clase almacena la imagen RGB original y su versión en escala de grises. A partir de ahí, se organiza en subclases funcionales, cada una dedicada a un conjunto específico de operaciones:

- **Contrast:** contiene los métodos:
 - `stretch()`, que realiza una expansión lineal del histograma para mejorar el contraste general de la imagen.
 - `equalize()`, que realiza la ecualización de histograma de una imagen de un canal.
- **ColorDetection:** agrupa métodos para generar máscaras basadas en la detección de color rojo:

- `red_mask_rgb()`: aplica un umbral directo sobre los canales R, G y B.
 - `dominant_red_mask()`: genera una máscara basada en la predominancia del canal rojo sobre los otros.
 - `red_mask_hsv()`: convierte la imagen a HSV y segmenta el rojo utilizando rangos de tono, lo que mejora la robustez frente a cambios de iluminación.
- **Thresholding:** incluye métodos para binarización de imágenes en escala de grises:
- `global_thresh()`: aplica un umbral fijo.
 - `adaptive_thresh()`: emplea umbralizado adaptativo, útil en condiciones de iluminación no homogénea.
- **Morphology:** contiene operaciones morfológicas clásicas para limpiar o realzar regiones de las máscaras:
- `open()`, `close()`, `erode()`, `dilate()`.
- **Visualization:** implementa el método `show_masks()`, que permite visualizar múltiples máscaras en una misma figura con títulos personalizados, lo cual facilita la comparación visual entre métodos y configuraciones.

Gracias a esta arquitectura, el preprocesamiento de imágenes se puede construir de forma flexible, combinando diferentes etapas de forma ordenada y reutilizable.

A continuación se muestran algunos resultados tras aplicar distintos preprocesados (máscara de HSV y las etapas del *pipeline* seleccionado).

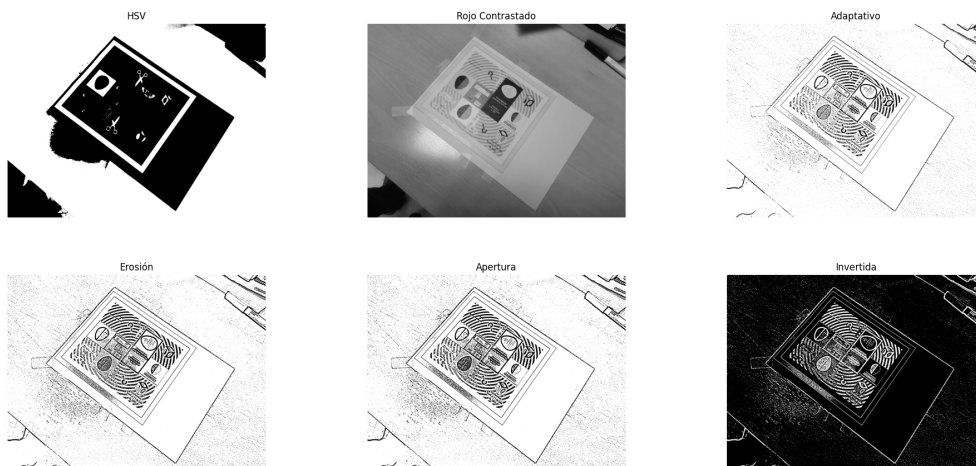


Figura 8: Ejemplos del preprocesado aplicado para segmentar el marco rojo en una misma escena.

3.2. Detección de contornos

El proceso de detección de contornos también ha sido complejo, porque dependía del paso anterior y era difícil estandarizar el proceso. A continuación, analizaremos los problemas que nos hemos encontrado y explicaremos el método que mejor nos ha funcionado.

3.2.1. Detección de contornos anidados

Aunque para calcular la homografía sería suficiente con las coordenadas del marco exterior, como en la guía de la práctica aparecían marcadas tanto las esquinas externas como las internas del marco rojo, se planteó como subobjetivo detectar ambas.

Para ello, utilizamos la función `cv2.findContours()`, que permite detectar contornos en imágenes binarias. Sus argumentos más relevantes son:

- **image:** imagen binaria (por ejemplo, una máscara).
- **mode:** modo de recuperación de contornos.² Algunos modos útiles son:
 - `cv2.RETR_EXTERNAL`: solo devuelve los contornos exteriores.
 - `cv2.RETR_LIST`: devuelve todos los contornos sin jerarquía.
 - `cv2.RETR_TREE`: devuelve todos los contornos e incluye su jerarquía (padres e hijos).
- **method:** método de aproximación. Por ejemplo:
 - `cv2.CHAIN_APPROX_SIMPLE`: elimina puntos redundantes.
 - `cv2.CHAIN_APPROX_NONE`: devuelve todos los puntos del contorno.

Esta función devuelve una lista de contornos y una jerarquía asociada. Más información disponible en la documentación oficial de OpenCV: [findContours - OpenCV Docs](#).

En nuestro caso, probamos inicialmente con `cv2.RETR_EXTERNAL`, pero dado que necesitábamos identificar contornos anidados (interior y exterior), optamos finalmente por `cv2.RETR_TREE`. Este modo organiza los contornos mediante la estructura: [Next, Previous, First_Child, Parent].

Una vez llamada la función:

```
contours, hierarchy = cv2.findContours(inverted, cv2.RETR_TREE,
    ↪ cv2.CHAIN_APPROX_SIMPLE)
```

Recorremos los contornos y filtramos los pares que nos interesan. En concreto, buscamos contornos que tengan un padre (es decir, que estén anidados), que tengan forma de

²https://docs.opencv.org/4.x/d9/d8b/tutorial_py_contours_hierarchy.html

cuadrilátero convexo y un área mínima, y que no tengan proporción de folio A4 (que aparece con frecuencia como contorno irrelevante). Esto se implementa en la función:

```
def find_nested_quads_by_hierarchy(contours, hierarchy, min_area=10000):
    nested_pairs = []
    for idx, h in enumerate(hierarchy[0]):
        parent_idx = h[3]
        if parent_idx != -1:
            child = contours[idx]
            parent = contours[parent_idx]
            if (cv2.contourArea(child) > min_area and
                cv2.contourArea(parent) > min_area):

                approx_child = cv2.approxPolyDP(child, 0.02 *
                    ↪ cv2.arcLength(child, True), True)
                approx_parent = cv2.approxPolyDP(parent, 0.02 *
                    ↪ cv2.arcLength(parent, True), True)

                if (len(approx_child) == 4 and
                    ↪ cv2.isContourConvex(approx_child) and
                    len(approx_parent) == 4 and
                    ↪ cv2.isContourConvex(approx_parent)):

                    if is_a4_ratio(approx_parent) or
                        ↪ is_a4_ratio(approx_child):
                        continue
                    nested_pairs.append((approx_parent, approx_child))
```



Figura 9: Pares de contornos anidados filtrados con `find_nested_quads_by_hierarchy`.

No obstante, observamos que en algunos casos se detectaban múltiples contornos casi idénticos para un mismo borde (véase Figura 9), posiblemente debido a irregularidades del preprocesado. Para mitigar este problema, implementamos un filtrado adicional que descarta pares demasiado similares, evaluando la distancia media entre sus esquinas. (También se valoró medir la similitud con la Intersección partido de la Union (IoU) por ser más robusta frente a distintos tamaños, pero no se realizó por falta de tiempo). La función utilizada para este caso práctico sería:

```
def filter_similar_quad_pairs(pairs, max_corner_dist=10):
    filtered = []
    for outer, inner in pairs:
        outer_ord = order_corners(outer)
        inner_ord = order_corners(inner)
        distances = np.linalg.norm(outer_ord - inner_ord, axis=1)
        mean_dist = np.mean(distances)
        if mean_dist > max_corner_dist:
            filtered.append((outer, inner))
```

Par filtrado



Figura 10: Par exterior e interior filtrado con `filter_similar_quad_pairs`.

Una vez realizado el filtrado, se obtienen las 8 esquinas (4 exteriores y 4 interiores), que pueden visualizarse como se muestra en la siguiente figura:

Con el marco exterior correctamente detectado y ordenado, ya es posible calcular la homografía utilizando:

```
H_t_img2, _ = cv2.findHomography(contour_outer_template, outer_ord,
    ↪ cv2.RANSAC)
```

donde `contour_outer_template` representa las coordenadas del marco en la plantilla y `outer_ord` las del marco rojo en la imagen, ambas ordenadas como: [top-left, top-right, bottom-right, bottom-left].

Esquinas ordenadas: exterior (verde), interior (azul)



Figura 11: Coordenadas del marco exterior e interior ordenadas.

Cabe destacar que, a pesar de este procedimiento, el método no resulta del todo robusto con el pipeline actual, ya que en algunas escenas no se detectan contornos anidados válidos, devolviendo None y haciendo fallar el sistema.